

מבנה מחשבים – תרגיל בית 1

חלק יבש

מגשים:

אביב אלימלך

ת"ז: 215248519

מייל: avive@campus.technion.ac.il

אילן בלווקה

ת"ז: 340875624

מייל: balouka.ilan@campus.technion.ac.il

שאלה 1:

א.

$$\begin{aligned}CPI &= \sum frequency_i * clock_cycles_i = \\(0.5 * 1) + (0.15 * 5) + (0.2 * 4) + (0.15 * 2) &= \\0.5 + 0.75 + 0.8 + 0.3 &= \mathbf{2.35}\end{aligned}$$

ב. ה-CPI ישאר זהה – הקטנה של זמן מחזור השעון, בתנאי שזה השינוי היחיד שנעשה במעבד, לא תגרום לשינוי בכמות הפקודות למחזור שעון, אלא רק לביצוע מהיר יותר של כל פקודה (בהנחה והמעבד עדיין עומד באילוצי זמנים וכל פקודה מספיקה לסיים את השלב שבו היא נמצאת בזמן קטן פי 2 ממוקדם) כולמר: $CPI = 2.35$

ג.

$$\begin{aligned}New\ CPI &= \sum frequency_i * clock_cycles_i = \\(0.5 * 1 * 1.5) + \left(0.15 * \frac{5}{2}\right) + \left(0.2 * \frac{4}{3}\right) + (0.15 * 2) &= \\0.75 + 0.375 + 0.266 + 0.3 &= \mathbf{1.691}\end{aligned}$$

$$\begin{aligned}CPU_{old} &= 2.35 * IC \\CPU_{new} &= \frac{1.691}{2} * IC = 0.846 * IC\end{aligned}$$

$$speedup = \frac{CPI_{old}}{CPI_{new}} = \frac{2.35 * IC}{0.846 * IC} = \sim 2.78 \text{ של קיבלנו שיפור}$$

ד. נחשב CPI בתוכנית מהקומפיילר החדש (תחת ההנחה שמשתמשים במעבד החדש עבור אותה התוכנית בשני הקומפיילרים):

$$CPI = \sum frequency_i * clock_cycles_i =$$

$$(0.3 * 1.5) + \left(0.15 * \frac{5}{2}\right) + \left(0.4 * \frac{4}{3}\right) + (0.15 * 2) = 0.45 + 0.375 + 0.533 + 0.3 = 1.658$$

קעת נחשב את זמן המעבד עבור שני הקומפילרים עם המשתנה $x = \text{instruction count} * \text{cycle time}$:

$$CPU_Time_1 = CPI_1 * x = 1.691x$$

$$CPU_Time_2 = CPI_2 * 0.8 * x = 1.326x$$

הקומפילר החדש עדיף לפי מדד ה-CPU Time מכיוון שהוא מריץ את התוכנית על המעבד החדש מהר יותר מאשר הקומפילר הרגיל.

שאלה 2:

א. הבעיה תקרה כאשר במחזור שעון מסוים פקודה אחת נמצאת בשלב WB ומנסה לכתוב לקובץ הרגיסטרים (נסמן כ"פקודה 1"), ובאותו מחזור פקודה אחרת נמצאת בשלב ID ומנסה לקרוא ממנו (נסמן כ"פקודה 2"). כלומר, שבאותו מחזור השעון המעבד יצטרך לכתוב בחזרה את הערך שחושב בפקודה (1) לקובץ הרגיסטרים וגם לקרוא מקובץ הרגיסטרים בשביל פקודה (2), מה שלפי נתוני השאלה לא אפשרי ולכן לא נוכל לבצע את הפקודות במקביל ונקבל structural hazard. פקודה 1 יכולה להיות R/I-type כמו xor או addi, אבל לא פקודת J-type משום שפקודות אלו אינן כותבות לרגיסטרים במהלך שלב ה-WB. הפקודה 2 שקוראת את הרגיסטרים יכולה גם להיות פקודת R/I-type כמו add, subi, וכו' אבל לא J-type משום שפקודות אלו לא קוראות מרגיסטרים.

ב. בדומה למקרה שנלמד בתרגול על Load Use Hazard, ה-HDU שלנו יגיב בצורה דומה: כאשר ה-HDU מזהה מצב שבו פקודה אחת נמצאת בשלב WB ומבצעת כתיבה לקובץ הרגיסטרים (פקודה 1), ובאותו מחזור פקודה אחרת נמצאת בשלב ID ומבקשת לקרוא ממנו (פקודה 2). במצב כזה נטפל בבעיה כך: נעשה assert gate על PC ועל IF/ID (נקפא אותם), נשלח 0 לשער ה-AND של יחידת הבקרה כך שהאותות הנכנסים ל-ID/EX יהיו אפס, ובכך נזריק NOP בין פקודה (2) לפקודה שקדמה לה. במהלך מחזור השעון הנוכחי ניתן עדיפות לביצוע WB של פקודה (1), נבצע כתיבה אל הרגיסטר שפקודה (1) כותבת אליו. רק במחזור השעון הבא פקודה (2) תקרא את ערכי האוגרים שלה מקובץ האוגרים ובאמת תבצע את שלב ה-ID. לבסוף נקבל ששתי הפקודות יתבצעו ללא בעיות ובסדר הנכון, אך נבזבז מחזור שעון אחד בין שתי הפקודות שבוא תצא פקודה ריקה/בועה (NOP).

ג. כדי להוציא את ה-CPI המקסימלי מהמעבד הנתון, אנחנו נרצה לגרום לכך שיהיה בתוכנית שלנו כמה שיותר structural hazard. ניצור אותם על ידי העמסה של פקודות שגם כותבות וגם קוראות מרגיסטרים כדי לאלץ את ה-HDU להכניס כמה שיותר בועות בשביל להוסיף יותר מחזורים לביצוע התוכנית הכולל. נחשוב מה קורה עבור תוכנית שמורכבת רק מפקודות add בלתי תלויות (כדי למנוע

(data hazard
לדוגמה:

```
add $t0, $s0, $s1
add $t1, $s2, $s3
add $t2, $s4, $s5
add $t3, $s6, $s7
add $t4, $a0, $a1
```

...

בכל מחזור לאחר שה pipeline טעון לגמרי – יש לנו WB ו ID פקודות שאחת צריכה לקרוא ואחרת לכתוב לקובץ הרגיסטרים. ה HDU יכניס בועה אחרי הפקודה שב ID ויקדם את הפקודות שקדמו לה. במחזור הבא – שוב יקרה אותו מצב כי הפקודה שב WB עדיין תהיה פקודה שכותבת בחזרה לרגיסטר, ולכן נכניס עוד בועה אחרי אותה הפקודה ששמרנו ב ID. בחזור שאחריו – שוב, מאותה הסיבה, עוד בועה אחרי ID כך שהשלבים EX, M, WB יהיו תפוסים על ידי בועות. לבסוף אחרי שתסתיים הפקודה שקדמה בתוכנית המקורית לפקודה שאנחנו עדיין מחזיקים ב ID, נוכל סוף סוף לקרוא מקובץ הרגיסטרים ולהמשיך. המעגל הזה יחזור על עצמו לאחר שה pipeline יתמלא שוב – כלומר בסך הכל נקבל:

$$Total\ Cycles = 4 + 3 + (i - 1) * (3 + 3) = 1 + 6i$$

כאשר i הוא מספר הבלוקים של 3 פקודות בתוכנית (נניח שהוא מספר שלם, כלומר שמספר הפקודות בתוכנית מתחלק ב 3) 4 מחזורים ראשונים ממלאים את הצינור, ואז לכל בלוק של 3 פקודות בתוכנית: 3 מחזורים יתבזבזו על בועות ו 3 על פקודות אמיתיות והזרקה של בועות נוספות. בסך הכל נקבל:

$$CPI = \frac{Total\ Cycles}{no.\ instructions} = \frac{1 + 6i}{3i} = \frac{1}{3i} + 2 \rightarrow_{i \rightarrow \infty} 2$$

ה CPI המקסימלי הוא 2.

שאלה 3:

א.

Tag	Set	Offset
11-63	6-10	0-5

גודל הבלוק הוא $2^6 = 64$ בתים, לכן יש צורך ב-6 ביטים של offset

$$\frac{8KB}{64} = 128 = 2^7 \text{ מספר הבלוקים במטמון הוא } 2^7$$

המטמון שלנו הוא 4-way ולכן מספר set'ים בכל way הוא: $2^5 = \frac{2^7}{4}$

לכן צריך 5 ביטים ל set

וכל שאר הביטים tag

ב.

X	L1	נימוק
1	$\frac{1}{12}$	נבחין בתכונה הבאה: התוכנית ניגשת תמיד ל- $A[i]$, ורק אם i זוגי היא ניגשת גם ל- $B[i]$. יש גישה ל- A בכל איטרציה, כל בלוק של A מכיל 16 איברים, ולכן יש $miss$ אחד על כל 16 גישות. ל- B ניגשים רק עבור אינדקסים זוגיים, גם כאן כל הגישות האלה נמצאות באותו בלוק של B , לכן יש $miss$ אחד לכל 8 גישות ל- B . בכל 16 איטרציות של הלולאה, יש 16 גישות ל- A ו-8 גישות ל- B (24 גישות בסך הכל) ולכן בכל 16 איטרציות יש $miss$ אחד מ- A ו- $miss$ אחד מ- B . סך הכל: $miss$ 2 על 24 גישות נותן לנו $miss\ rate = 2/24 = 1/12$
16	1	נשים לב שאנו ניגשים לאיברים $A[16], A[32], \dots$ ושהם כולם בבלוקים שונים, לכן כל גישה ל- A תהיה $miss$. עכשיו נסתכל על B . מכיוון שכל האינדקסים הם כפולות של 16, הם כולם זוגיים, ולכן התנאי ב- if תמיד מתקיים ולכן ניגש ל- B . גם כאן כל גישה ל- B היא $miss$ מאותה הסיבה של A . לכן בכל גישה שלנו לאחד המערכים אנחנו נתקלים ב- $miss$ ולכן: $miss\ rate = 1$
32	1	עבור $x=32$ המצב כמעט זהה ל- $x=16$ אך עם ניצול אפילו יותר גרוע של המטמון. גם פה כל גישה ל- A היא לבלוק חדש שמעולם לא השתמשנו בו קודם ולכן תתן $miss$. בנוסף, כל האינדקסים הם זוגיים, ולכן תמיד ניכנס ל- if וניגש גם ל- B , ששם גם כן כל גישה היא לבלוק חדש שיתן $miss$ סך הכל – $miss\ rate = 1$ שוב
2^{16}	1	ניתן בקלות לשים לב שזוהי גם כפולה של 16 ולכן גם היא תסבול מאותן הבעיות של $x=16$ ו- $x=32$. מכיוון שכל גישה ל- A או ל- B היא בהכרח לבלוק חדש שלא שומש קודם כל גישה כזאת תתן $miss$ $miss\ rate = 1$

ג.

Tag	Offset
6-63	0-5

ב-*fully associative* אין שדה *Set*, כי כל בלוק יכול להיכנס לכל שורה במטמון.
גודל כתובת הוא 64 ביטים עדיין ולכן עדיין צריך 6 ביטים של *offset*.
וכל שאר הביטים יהיו *tag*.

ביצועי התוכנית הנתונה עם השינוי במטמון לאו דווקא ישתפרו ברוב המקרים.
אם ה-*misses* בתוכנית נובעים בעיקר מ-*compulsory misses* / חוסר לוקאליות במרחב בגלל קפיצות גדולות ב- x , אז *fully associative* לא יעזור. הוא עוזר בעיקר להפחית *conflict misses*.